Table 4.1 Cloud native applications can be configured according to different strategies. You will likely use them all, depending on the type of configuration data and the application requirements.

Configuration strategy	Characteristics
Property files packaged with the application	• These files can act as specifications of what configuration data the application supports. • These are useful for defining sensible default values, mainly oriented to the development environment.
Environment variables	• Environment variables are supported by any operating system, so they are great for portability. • Most programming languages allow you access to the environment variables. In Java you can access them with the `System.getenv()` method. In Spring you can also rely on the `Environment` abstraction. • These are useful for defining configuration data that depends on the infrastructure and platform where the application is deployed, such as active profiles, hostnames, service names, and port numbers.
Configuration service	• Provides configuration data persistence, auditing, and accountability. • Allows secrets management by using encryption or dedicated secret vaults. • This is useful for defining configuration data specific to the application, such as connection pools, credentials, feature flags, thread pools, and URLs to third-party services.